

Math Engine Project

Supervision: Dr. Nermeen Elkot

Contributors:

1-

2-

3-

4-

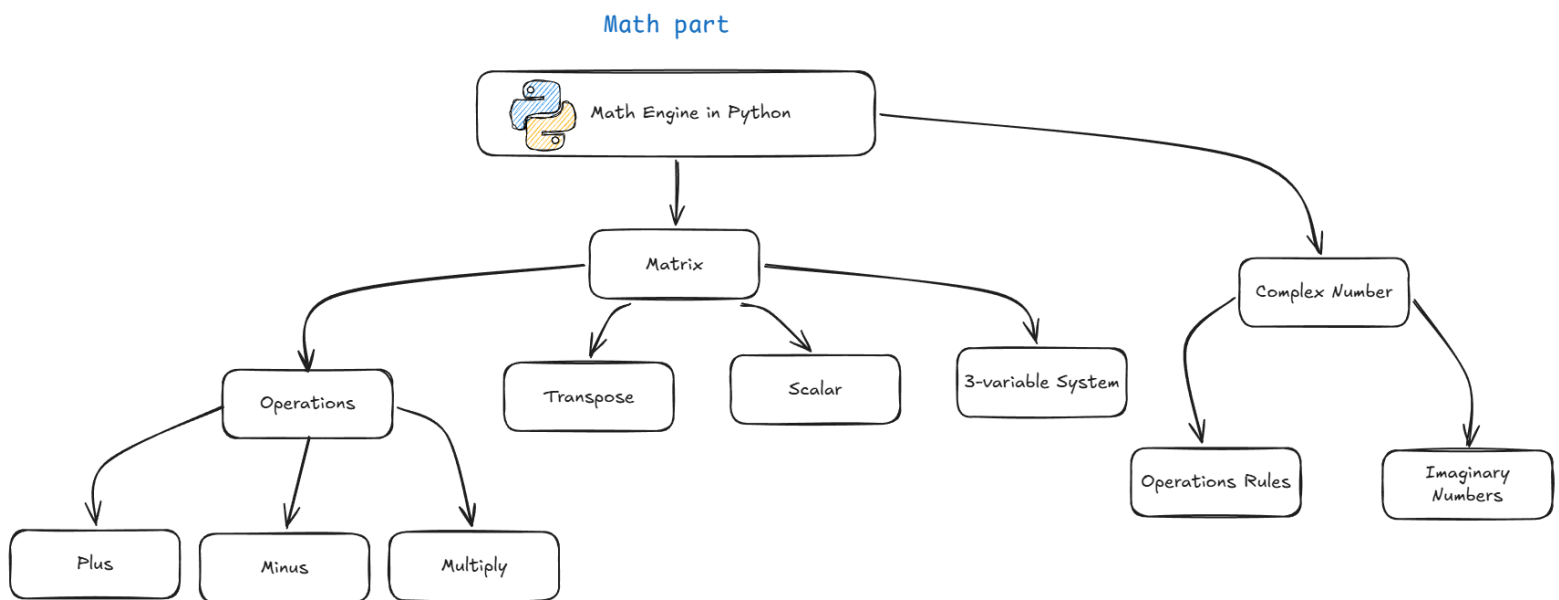
5-

Project Aim:

This project aims to create a Desktop based software to solve Mathematical Equations related to Matrices and Complex Numbers.

Project working Scope:

- Math constrains:

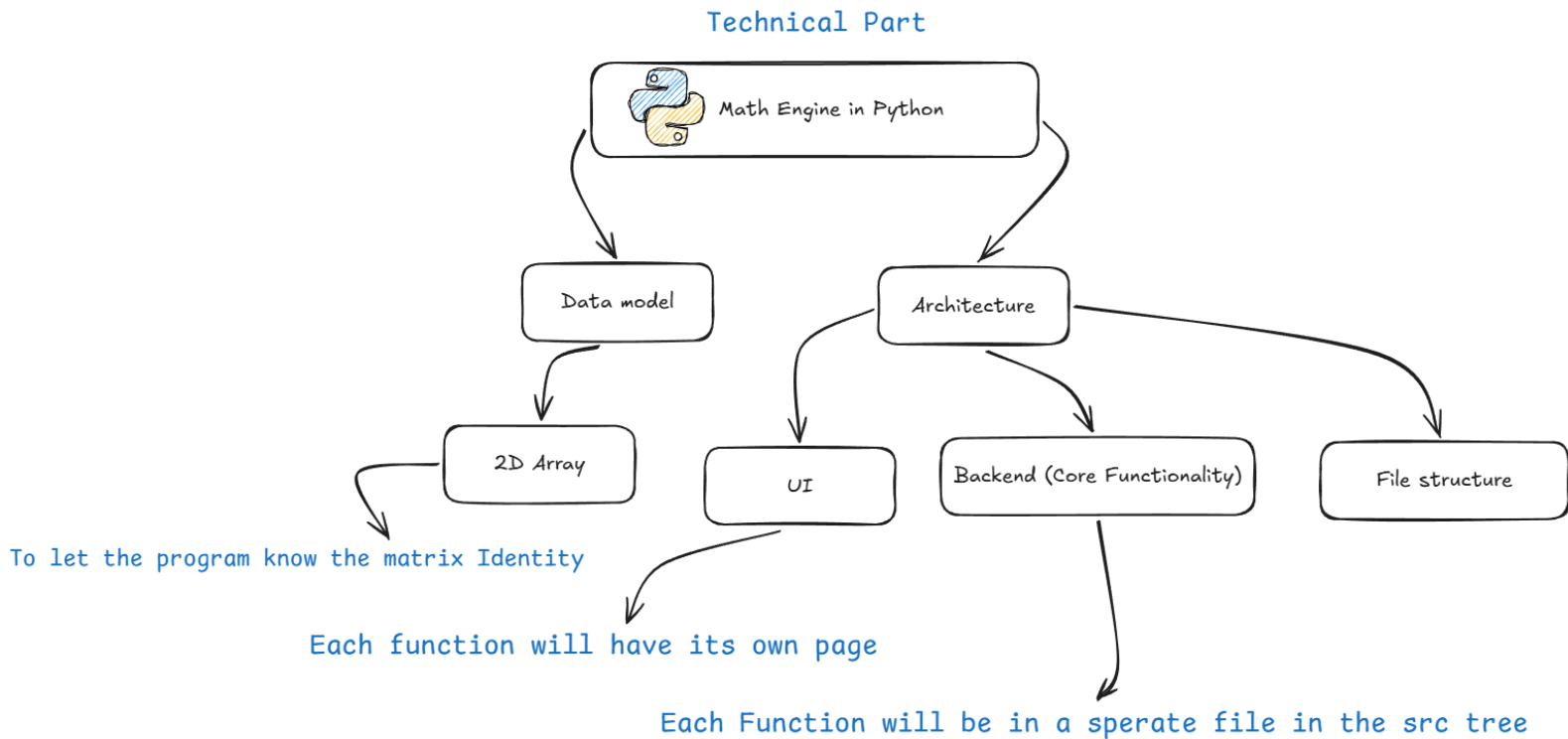


We needed to full understand the identity of the math operations done in each sector of the Desktop software operations like:

- 1- Matrix addition / Subtraction rules
- 2- Matrix multiplication rules
- 3- Complex numbers conjugate rule etc....

Each of these sectors was needed to be understood clearly to start working on the project

- Technical Constrains:



The Desktop based software will be built using python by (math, pyside6) libraries
For the mathematical operations and the GUI.

Code for the Mathematical Equations:

- Matrix:

```
@staticmethod
def binary_op(A: Matrix, B: Matrix, op: str) -> Result:
    t0 = _t()
    rA, cA = len(A), len(A[0])
    rB, cB = len(B), len(B[0])

    if op in ("add", "sub", "hadamard"):
        if rA != rB or cA != cB:
            return Result(ok=False, error=f"Dimension mismatch: [{rA}x{cA}] vs [{rB}x{cB}]")
        elif op == "mul":
            if cA != rB:
                return Result(ok=False, error=f"Cannot multiply [{rA}x{cA}] · [{rB}x{cB}] – cols(A) must equal rows(B)")

            if op == "add":
                C = [[A[i][j] + B[i][j] for j in range(cA)] for i in range(rA)]
                dims = f"{rA}x{cA}"
            elif op == "sub":
                C = [[A[i][j] - B[i][j] for j in range(cA)] for i in range(rA)]
                dims = f"{rA}x{cA}"
            elif op == "mul":
                C = [[sum(A[i][k] * B[k][j] for k in range(cA)) for j in range(cB)] for i in range(rA)]
                dims = f"{rA}x{cB}"
            elif op == "hadamard":
                C = [[A[i][j] * B[i][j] for j in range(cA)] for i in range(rA)]
                dims = f"{rA}x{cA}"
            else:
                return Result(ok=False, error=f"Unknown operation: {op}")

    return Result(ok=True, matrix=C, dims=dims, elapsed_ms=(t() - t0) * 1000)
```

— Transpose —————

```
@staticmethod
def transpose(A: Matrix) -> Result:
    t0 = _t()
    rows, cols = len(A), len(A[0])
    T = [[A[i][j] for i in range(rows)] for j in range(cols)]
    return Result(ok=True, matrix=T, dims=f"{cols}x{rows}", elapsed_ms=(_t() - t0) *
1000)
```

— 3-Variable Linear System —————

```
@staticmethod
def solve_system3(A: Matrix, b: list[float]) -> Result:
    t0 = _t()
    n = 3
    aug = [A[i][:] + [b[i]] for i in range(n)]
    for col in range(n):
        mx = max(range(col, n), key=lambda r: abs(aug[r][col]))
        aug[col], aug[mx] = aug[mx], aug[col]
        if abs(aug[col][col]) < 1e-12:
            return Result(ok=False, error="Singular system – no unique solution exists")
        for row in range(col + 1, n):
            f = aug[row][col] / aug[col][col]
            for j in range(col, n + 1):
                aug[row][j] -= f * aug[col][j]
    x = [0.0] * n
    for i in range(n - 1, -1, -1):
        x[i] = aug[i][n]
        for j in range(i + 1, n):
            x[i] -= aug[i][j] * x[j]
        x[i] /= aug[i][i]
    return Result(ok=True, vector=x, elapsed_ms=(_t() - t0) * 1000)
```

— Scalar Multiply —————

```
@staticmethod
def scalar_multiply(k: float, A: Matrix) -> Result:
    t0 = _t()
    rows, cols = len(A), len(A[0])
    C = [[k * A[i][j] for j in range(cols)] for i in range(rows)]
    return Result(ok=True, matrix=C, dims=f"{rows}x{cols}", elapsed_ms=(_t() - t0) *
1000)
```

— Inverse

```
@staticmethod
def inverse(A: Matrix) -> Result:
    t0 = _t()
    n = len(A)
    aug = [A[i][:] + [1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
    for col in range(n):
        mx = max(range(col, n), key=lambda r: abs(aug[r][col]))
        aug[col], aug[mx] = aug[mx], aug[col]
        if abs(aug[col][col]) < 1e-12:
            return Result(ok=False, error="Matrix is singular – inverse does not exist")
        pv = aug[col][col]
        aug[col] = [v / pv for v in aug[col]]
        for row in range(n):
            if row == col:
                continue
            f = aug[row][col]
            aug[row] = [aug[row][j] - f * aug[col][j] for j in range(2 * n)]
    inv = [row[n:] for row in aug]
    return Result(ok=True, matrix=inv, dims=f"{n}x{n}", elapsed_ms=(_t() - t0) * 1000)
```

Complex:

```
class ComplexEngine:
    """Pure Python complex number operations engine."""

    @staticmethod
    def add(a: float, b: float, c: float, d: float) -> ComplexResult:
        """Add two complex numbers: (a+bi) + (c+di)"""
        t0 = _t()
        real = a + c
        imag = b + d
        return ComplexEngine._build_result(real, imag, "addition", t0)

    @staticmethod
    def subtract(a: float, b: float, c: float, d: float) -> ComplexResult:
        """Subtract two complex numbers: (a+bi) - (c+di)"""
        t0 = _t()
        real = a - c
        imag = b - d
        return ComplexEngine._build_result(real, imag, "subtraction", t0)

    @staticmethod
    def multiply(a: float, b: float, c: float, d: float) -> ComplexResult:
        """Multiply two complex numbers: (a+bi) × (c+di)"""
        t0 = _t()
        real = a * c - b * d
        imag = a * d + b * c
        return ComplexEngine._build_result(real, imag, "multiplication", t0)

    @staticmethod
    def divide(a: float, b: float, c: float, d: float) -> ComplexResult:
        """Divide two complex numbers: (a+bi) ÷ (c+di)"""
        t0 = _t()
        denominator = c * c + d * d
        if denominator == 0:
            return ComplexResult(
                ok=False,
                error="Division by zero – denominator cannot be 0+0i",
                operation="division",
                elapsed_ms=(_t() - t0) * 1000
            )
        real = (a * c + b * d) / denominator
        imag = (b * c - a * d) / denominator
```

```
    return ComplexEngine._build_result(real, imag, "division", t0)
```

```
@staticmethod
```

```
def conjugate(a: float, b: float) -> ComplexResult:
```

```
    """Compute conjugate:  $a - bi$ """
```

```
    t0 = _t()
```

```
    real = a
```

```
    imag = -b
```

```
    return ComplexEngine._build_result(real, imag, "conjugate", t0)
```

```
@staticmethod
```

```
def modulus(a: float, b: float) -> ComplexResult:
```

```
    """Compute modulus:  $|z| = \sqrt{a^2 + b^2}$ """
```

```
    t0 = _t()
```

```
    mod = math.sqrt(a * a + b * b)
```

```
    return ComplexEngine._build_result(mod, 0.0, "modulus", t0)
```

```
@staticmethod
```

```
def argument(a: float, b: float) -> ComplexResult:
```

```
    """Compute argument:  $\theta = \text{atan2}(b, a)$ """
```

```
    t0 = _t()
```

```
    arg = math.atan2(b, a)
```

```
    return ComplexEngine._build_result(arg, 0.0, "argument", t0)
```

```
@staticmethod
```

```
def to_polar(a: float, b: float) -> ComplexResult:
```

```
    """Convert to polar form:  $r(\cos \theta + i \sin \theta)$ """
```

```
    t0 = _t()
```

```
    r = math.sqrt(a * a + b * b)
```

```
    theta = math.atan2(b, a)
```

```
    return ComplexEngine._build_polar_result(a, b, r, theta, t0)
```

```
@staticmethod
```

```
def _build_result(real: float, imag: float, operation: str, t0: float) -> ComplexResult:
```

```
    """Build a ComplexResult with all computed properties."""
```

```
    modulus = math.sqrt(real * real + imag * imag)
```

```
    argument = math.atan2(imag, real)
```

```
    conjugate_real = real
```

```
    conjugate_imag = -imag
```

```
    # For polar form of result
```

```
    polar_r = modulus
```

```
    polar_theta = argument
```



```

return ComplexResult(
    ok=True,
    result_real=real,
    result_imag=imag,
    modulus=modulus,
    argument=argument,
    conjugate_real=conjugate_real,
    conjugate_imag=conjugate_imag,
    polar_r=polar_r,
    polar_theta=polar_theta,
    operation=operation,
    elapsed_ms=(_t() - t0) * 1000
)

```

```

@staticmethod
def _build_polar_result(a: float, b: float, r: float, theta: float, t0: float) ->
ComplexResult:
    """Build a ComplexResult specifically for polar form conversion."""
    modulus = r
    argument = theta
    conjugate_real = a
    conjugate_imag = -b

    return ComplexResult(
        ok=True,
        result_real=r,
        result_imag=theta,
        modulus=modulus,
        argument=argument,
        conjugate_real=conjugate_real,
        conjugate_imag=conjugate_imag,
        polar_r=r,
        polar_theta=theta,
        operation="polar_form",
        elapsed_ms=(_t() - t0) * 1000
    )

```

```

@staticmethod
def format_complex(real: float, imag: float) -> str:
    """Format a complex number as a string."""
    if imag >= 0:
        return f"{fmt(real)} + {fmt(imag)}i"

```

```
else:  
    return f"{fmt(real)} - {fmt(abs(imag))}i"
```